

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №2
по дисциплине «Методы тестирования программного
обеспечения»
на тему «Тестирование методом чёрного и белого ящика»

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Курочкин М. А. _____

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Описание методов тестирования	6
2.1 Метод чёрного ящика	6
2.1.1 Эквивалентное разбиение	6
2.1.2 Анализ граничных значений	7
2.1.3 Применение причинно-следственных диаграмм	7
2.2 Метод белого ящика	10
2.2.1 Покрытие операторов	10
2.2.2 Покрытие решений	10
2.2.3 Покрытие условий	10
2.2.4 Покрытие решений и условий	10
2.2.5 Комбинаторное покрытие условий	10
3 Тестирование программы №1	11
3.1 Блок-схема алгоритма	11
3.2 Тестирование методом белого ящика	13
3.2.1 Покрытие операторов	13
3.2.2 Покрытие решений	14
3.2.3 Покрытие условий	14
3.2.4 Покрытие решений и условий	14
3.2.5 Комбинаторное покрытие условий	15
3.2.6 Результаты тестирования методом белого ящика	15
3.3 Тестирование методом чёрного ящика	15
3.3.1 Разбиение на классы эквивалентности	15
3.3.2 Анализ граничных значений	16
3.3.3 Причинно-следственная диаграмма	17
3.3.4 Результаты тестирования методом черного ящика	18
4 Тестирование программы №2	19
4.1 Блок-схема алгоритма	19
4.2 Тестирование методом белого ящика	21
4.2.1 Покрытие операторов	21
4.2.2 Покрытие решений	21
4.2.3 Покрытие условий	21
4.2.4 Покрытие решений и условий	22
4.2.5 Комбинаторное покрытие условий	22
4.2.6 Результаты тестирования методом белого ящика	23
4.3 Тестирование методом чёрного ящика	23
4.3.1 Разбиение на классы эквивалентности	23

4.3.2	Анализ граничных условий	24
4.3.3	Причинно-следственная диаграмма	25
4.3.4	Результаты тестирования методом черного ящика	27
Заключение		28

Введение

Модульное тестирование — это процесс тестирования отдельных блоков, подпрограмм, классов или процедур, образующих крупную программу. Другими словами, прежде чем тестировать программу в целом, необходимо сосредоточить внимание на ее меньших по размеру компонентах.

Цель модульного тестирования — сравнение функций, реализуемых модулем, со спецификациями, описывающими его функциональные характеристики.

В данной работе будут рассмотрены два основных подхода модульного тестирования: методы «чёрного» и «белого» ящика.

1 Постановка задачи

Цель работы: провести тестирование двух программ методами чёрного и белого ящика.

Задачи

1. Изучить методы модульного тестирования, в частности методы чёрного и белого ящика.
2. Разработать тесты для двух программ.
3. Провести тестирование двух программ.
4. Проанализировать результаты тестирования программ.

2 Описание методов тестирования

2.1 Метод чёрного ящика

Тестирование *методом чёрного ящика* также называют *тестированием, управляемым данными*, или *тестированием, управляемым входом и выходом*. В соответствии с этим методом программа рассматривается как «чёрный ящик», внутреннее поведение и структура которого не имеют значения. Внимание фокусируется на выяснении обстоятельств, при которых поведение программы не соответствует спецификации.

Тестовые данные выбираются исключительно на основе спецификации требований, без привлечения знаний о внутренней структуре программы.

Далее будут рассмотрены три прёма тестирования методом чёрного ящика.

2.1.1 Эквивалентное разбиение

Исчерпывающее тестирование программы на всех возможных входных наборах данных неосуществимо. Следовательно, необходимо выбрать самое подходящее подмножество тестов, которое обеспечит наибольшую вероятность обнаружения большинства ошибок.

Хорошо спроектированный тест должен обладать двумя свойствами:

- уменьшать более чем на единицу число других тестов, которые должны быть разработаны для достижения поставленной цели;
- покрывать значительную часть других возможных тестов, то есть представлять некую информацию относительно наличия или отсутствия ошибок в ситуациях, не охватываемых данным конкретным набором входных значений.

Первое свойство означает, что каждый тест должен охватывать как можно большее число различных ситуаций на входе программы, чтобы свести к минимуму общее количество необходимых тестов. Второе свойство означает, что следует пытаться разбить всю область входных данных программы на конечное число *классов эквивалентности*. Это позволит с достаточной долей уверенности полагать, что тестирование представительного значения данного класса эквивалентно тестированию любого другого значения, принадлежащего тому же классу.

Определение классов эквивалентности сводится к последовательному рассмотрению каждого из входных условий и разбиению его на две или несколько групп. Определяется два типа классов эквивалентности: *допустимые* и *недопустимые классы эквивалентности*. Допустимые классы представляют допустимые входные данные программы, а недопустимые представляют все остальные возможные состояния условий.

Второй этап заключается в использовании классов эквивалентности для определения тестов. Этот процесс состоит из следующих этапов.

1. Назначить каждому классу эквивалентности уникальный номер.
2. Записывать новые тесты, включающие в себя как можно большее количество оставшихся неохваченными допустимых классов эквивалентности, пока не будут покрыты все допустимые классы.
3. Записывать новые тесты, каждый из которых охватывает один и только один из оставшихся неохваченными недопустимых классов эквивалентности, пока не будут покрыты все недопустимые классы.

2.1.2 Анализ граничных значений

Граничные условия – это ситуации, возникающие в области граничных значений входных и выходных классов эквивалентности. Анализ граничных значений отличается от методики разбиения на классы эквивалентности в двух отношениях.

1. Вместо того, чтобы выбирать любой элемент класса эквивалентности в качестве представителя всего класса, анализ граничных значений требует выбирать такой элемент или элементы, которые обеспечивают тестирование каждой границы класса.
2. При создании тестов внимание фокусируется не только на входных условиях, но и на пространстве результатов (выходные классы эквивалентности).

2.1.3 Применение причинно-следственных диаграмм

Причинно-следственная диаграмма представляет собой формальный язык, на который транслируется спецификация, написанная на естественном языке.

Для построения тестов используется процесс, включающий несколько этапов.

1. Спецификация разбивается на части, с которыми легче работать. Это приходится делать, поскольку причинно-следственные диаграммы для больших спецификаций разрастаются до громадных размеров.
2. В спецификации определяются причины и следствия. Причина — это отдельное входное условие или класс эквивалентности входных условий. Следствие — это выходное условие или преобразование системы. Причины и следствия определяются путем последовательного чтения спецификации и подчеркивания тех слов или фраз, которые описывают

причины и следствия. Каждой причине и каждому последствию присваивается уникальный номер.

3. Семантическое содержание спецификации анализируется и преобразуется в булев граф, связывающий причины и следствия. Полученный граф называется причинно-следственной диаграммой.
4. Диаграмма снабжается примечаниями, задающими ограничения и описывающими комбинации причин и (или) следствий, реализация которых невозможна из-за синтаксических или внешних ограничений.
 - (a) Ограничение E требует, чтобы всегда выполнялось условие, в соответствие с которым a и b не могут быть равны 1 одновременно, но обе величины могут быть равны 0 одновременно.
 - (b) Ограничение I требует, чтобы по крайней мере одна из величин a , b или c была равна 1.
 - (c) Ограничение O требует, чтобы одна и только одна из величин a или b была равна 1.
 - (d) Ограничение R требует, чтобы a было равно 1 только если b равно 1. Символы ограничений приведены на рис. 2.
5. Путем прослеживания состояний условий диаграмма преобразуется в таблицу решений с ограниченными входами. Каждый столбец таблицы решений соответствует тесту.
6. Столбцы таблицы решений преобразуются в тесты.

Базовая нотация причинно-следственных диаграмм представлена на рис.

1.

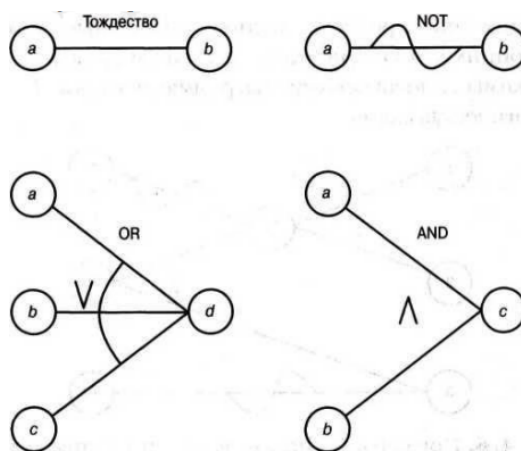


Рис. 1: Базовая нотация, используемая в причинно-следственных диаграммах

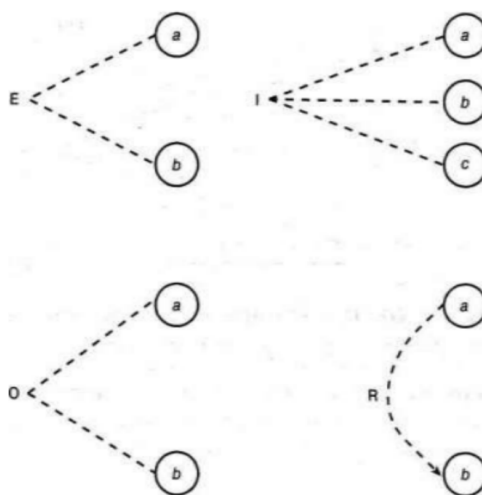


Рис. 2: Символы ограничений

Алгоритм заполнения очередного столбца таблицы решений

1. Выбрать следствие, которое должно находиться в состоянии 1.
2. Продвигаясь вдоль линий диаграммы, ведущих к данному узлу, в обратном направлении, производится поиск комбинаций причин (подчиненных ограничениям), которые устанавливают данное следствие в 1.
 - Если путь обратной трассировки проходит через узел типа OR, выход которого должен принимать значение 1, то не следует одновременно устанавливать в 1 более одного входа в этот узел.
 - Если путь обратной трассировки проходит через узел типа AND, выход которого должен принимать значение 0, то, конечно же, должны быть перечислены все комбинации входов, приводящие к установке выхода в 0. Однако при исследовании ситуации, в которой один вход равен 0, а один или несколько других входов равны 1, перечислять все условия, при которых другие входы находятся в состоянии 1, не обязательно.
 - Если путь обратной трассировки проходит через узел типа AND, выход которого должен принимать значение 0, то необходимо указать лишь одно условие, при котором все входы являются нулями.
3. Для каждой комбинации причин создается столбец в таблице решений.
4. Определить для каждой комбинации состояния всех других следствий и поместить их в соответствующий столбец таблицы.

2.2 Метод белого ящика

В стратегии тестирования *методом белого ящика*, которое называется также *тестированием, управляемым логикой программы*, разрешается исследовать внутреннюю структуру программы. Исходя из этой стратегии, тестирующий подбирает тестовые данные путем анализа логики программы.

Далее приведены методы «белого ящика».

2.2.1 Покрытие операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Это необходимое, но недостаточное условие для приемлемого тестирования по принципу белого ящика.

2.2.2 Покрытие решений

В соответствие с критерием покрытия решений количество тестов должно быть таким, чтобы каждое условие в программе хотя бы один раз принимало значение true и false. То есть каждая логическая ветвь каждой инструкции ветвления должна быть выполнена хотя бы один раз.

2.2.3 Покрытие условий

Количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное выражение в точке ветвления, принимало каждое из значений true и false по крайней мере один раз.

2.2.4 Покрытие решений и условий

Набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз. Каждый возможный исход решения также проверяется по крайней мере один раз и каждой такой точке входа управление передается по крайней мере один раз.

2.2.5 Комбинаторное покрытие условий

Этот критерий требует создания такого количества тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз.

3 Тестирование программы №1

Автор: Юлия Гордиенко

Название: Проверка числа на принадлежность к числам Армстронга

Дано:

- n – целое положительное число.

Требуется:

Определить, равно ли число сумме своих цифр, возведённых в степень, равную количеству цифр.

Ограничения:

1. n является целым числом в диапазоне $[0; 10^6]$.

Спецификация:

Таблица 1: Спецификация функции проверки числа Армстронга

Входные данные	Ожидаемый результат	Комментарий
$n = 0$ $n = 1$ $n = 5$	true true true	$0^1 = 0$ $1^1 = 1$ $5^1 = 5$
$n = 10$ $n = 153$ $n = 370$ $n = 123$	false true true false	$1^2 + 0^2 = 1 \neq 10$ $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ $3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370$ $1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36 \neq 123$
$n = 1634$ $n = 9474$	true true	$1^4 + 6^4 + 3^4 + 4^4 = 1 + 1296 + 81 + 256 = 1634$ $9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474$
$n = 1000000$ $n = -1$ $n = 1000001$	false false false	Число вне диапазона $[0; 10^6]$ Отрицательное число вне диапазона Число больше максимального

3.1 Блок-схема алгоритма

Блок-схема алгоритма приведена на рис. 3.

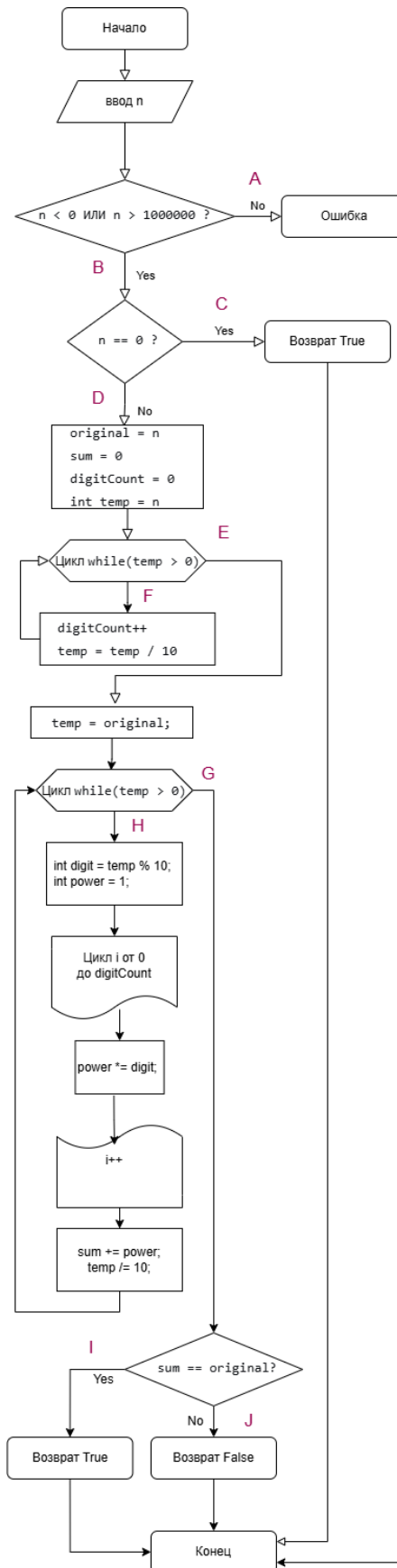


Рис. 3: Блок-схема алгоритма №1

3.2 Тестирование методом белого ящика

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы и проверку выполнения каждого оператора не менее одного раза. Для этого обозначим на блок-схеме алгоритма всевозможные пути, порожденные блоками условия, символами А-Ј.

В описанной программе можно выделить следующие ключевые условия и соответствующие им ветви.

1. $(n < 0) \vee (n > 10^6)$

- (a) Ветвь А: выход из программы с ошибкой размера.
- (b) Ветвь В: продолжение выполнения.

2. $n == 0$

- (a) Ветвь С: число принадлежит числам Армстронга, возвращение True.
- (b) Ветвь D: продолжение выполнения.

3. $temp > 0$ (условие цикла `while`)

- (a) Ветвь Е: выход из цикла.
- (b) Ветвь F: выполнение тела цикла (`digitCount++`, `temp = temp / 10`).

4. $temp > 0$ (условие второго цикла `while`):

- (a) G: выход из цикла.
- (b) Ветвь H: выполнение тела цикла (извлечение цифры, возведение в степень, добавление к `sum`).

5. $sum == original$

- (a) Ветвь I: возврат True.
- (b) Ветвь J: возврат False.

3.2.1 Покрытие операторов

Для покрытия всех операторов достаточно одного теста, который пройдет по основному пути и не вызовет досрочных завершений. В таблице 2 приведены результаты прохождения теста.

Тест №1 выполнил все операторы программы.

Таблица 2: Покрытие операторов

№	n	Ожид. рез.	Факт. рез.	Путь	Условия					Статус
					1	2	3	4	5	
1	153	True	True	BDFEHJI	F	F	T, F	T, F	T	Пройден

3.2.2 Покрытие решений

Необходимо составить такой набор тестов, при котором каждое условие в программе примет как истинное, так и ложное значение.

Тесты, покрывающие все решения программы, представлены в табл. 3.

Таблица 3: Покрытие решений

№	n	Ожид. рез.	Факт. рез.	Путь	Условия					Статус
					1	2	3	4	5	
1	153	True	True	BDFEHJI	F	F	T, F	T, F	T	Пройден
2	-1	Ошибка	Ошибка	A	T	-	-	-	-	Пройден
3	0	True	True	BC	F	T	-	-	-	Пройден
4	10	False	False	BDFEHGJ	F	F	T, F	T, F	F	Пройден

3.2.3 Покрытие условий

В соответствии с критерием покрытия условий количество тестов должно быть таким, чтобы все возможные результаты каждого условия в решении выполнялись как минимум один раз. В табл. 4 приведены тесты для покрытия условий.

Таблица 4: Покрытие условий

№	n	Ожид. рез.	Факт. рез.	Путь	Условия					Статус
					1	2	3	4	5	
1	153	True	True	BDFEHJI	(F, F)	F	T, F	T, F	T	Пройден
2	-1	Ошибка	Ошибка	A	(T, F)	-	-	-	-	Пройден
3	1000001	Ошибка	Ошибка	A	(F, T)	-	-	-	-	Пройден
4	0	True	True	BC	(F, F)	T	-	-	-	Пройден
5	10	False	False	BDFEHGJ	(F, F)	F	T, F	T, F	F	Пройден

3.2.4 Покрытие решений и условий

Набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решений повторяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз. В табл. 5 приведены тесты для покрытия решений и условий.

Таблица 5: Покрытие решений и условий

№	n	Ожид. рез.	Факт. рез.	Путь	Условия					Статус
					1	2	3	4	5	
1	-1	Ошибка	Ошибка	A	(T, F)	-	-	-	-	Пройден
2	1000001	Ошибка	Ошибка	A	(F, T)	-	-	-	-	Пройден
3	0	True	True	BC	F	T	-	-	-	Пройден
4	153	True	True	BDFEHGI	F	F	T, F	T, F	T	Пройден
5	10	False	False	BDFEHGJ	F	F	T, F	T, F	F	Пройден

3.2.5 Комбинаторное покрытие условий

Для комбинаторного покрытия условий требуется создать такой набор тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз. В табл. 6 приведены тесты для комбинаторного покрытия условий.

Таблица 6: Комбинаторное покрытие условий

№	n	Ожид. рез.	Факт. рез.	Путь	Условия					Статус
					1	2	3	4	5	
1	-1	Ошибка	Ошибка	A	(T, F)	-	-	-	-	Пройден
2	1000001	Ошибка	Ошибка	A	(F, T)	-	-	-	-	Пройден
3	0	True	True	BC	F	T	-	-	-	Пройден
4	153	True	True	BDFEHGI	F	F	T, F	T, F	T	Пройден
5	10	False	False	BDFEHGJ	F	F	T, F	T, F	F	Пройден

3.2.6 Результаты тестирования методом белого ящика

В ходе тестирования программы №1 методом белого ящика было разработано 5 тестов, на основе которых было составлено 5 тестовых наборов для покрытия операторов, решений, условий, решений и условий, а также комбинаторного покрытия.

Все тесты, построенные для программы №1, были успешно пройдены. Это свидетельствует о том, что мне, как неопытному тестировщику, не удалось подобрать набор тестов, при котором ошибки были бы обнаружены.

3.3 Тестирование методом чёрного ящика

3.3.1 Разбиение на классы эквивалентности

Из ограничений для программы составлено разбиение на классы эквивалентности, приведенные в табл. 7.

В таблице 8 приведены тесты, покрывающие классы эквивалентности.

Таблица 7: Классы эквивалентности для n

Вх. условие	Допустимые классы	Недопустимые классы
n	Число Армстронга (1)	Не число Армстронга (2), $n < 0$ (3), $n > 100000$ (4), вещественное число (5), не число (6)

Таблица 8: Тесты, покрывающие классы эквивалентности

№	n	Кл. экв-ти	Ожид. рез.	Факт. рез.	Статус
1	153	1	true	true	Пройден
2	100	2	false	false	Пройден
3	-5	3	Ошибка	Ошибка	Пройден
4	2000000	4	Ошибка	Ошибка	Пройден
5	10.5	5	Ошибка	Ошибка	Пройден
6	err	6	Ошибка	Ошибка	Пройден

3.3.2 Анализ граничных значений

Согласно спецификации, входные значения должны являться целыми числами, находящимися в диапазоне $[0, 1000000]$. Значит, можно выделить следующие граничные условия:

- $n \leq 0$;
- $n \geq 1000000$

Для каждой из границ определим тесты, соответствующие:

- граничному целому числу (верхнему/нижнему);
- целому числу, выходящему за верхнюю/нижнюю границу.

В таблице 9 приведены тесты, покрывающие проверку граничных условий.

Таблица 9: Тесты, покрывающие проверку граничных условий

№	n	Ожид. рез.	Факт. рез.	Статус
1	0	true	true	Пройден
2	1	true	true	Пройден
3	-1	Ошибка	Ошибка	Пройден
4	1000000	false	false	Пройден
5	999999	false	false	Пройден
6	1000001	Ошибка	Ошибка	Пройден

3.3.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введены следующие обозначения.

Причины.

1. n – целое.
2. $0 \leq n \leq 1000000$.
3. n равно сумме цифр в степени количества цифр (условие Армстронга).

Следствия.

4. Программа возвращает **true**.
5. Программа возвращает **false**.
6. Программа возвращает ошибку.

На рис. 4 приведена причинно-следственная диаграмма для рассматриваемой программы.

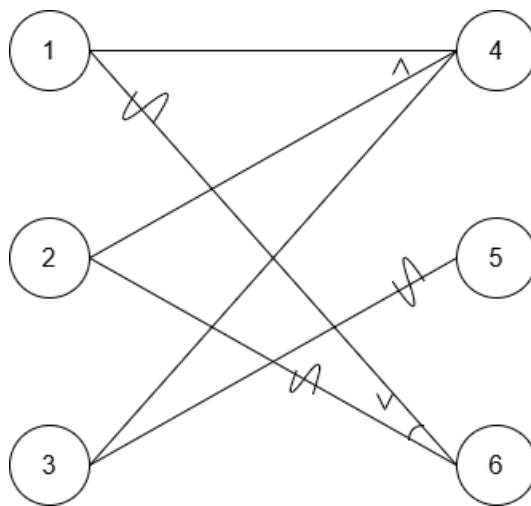


Рис. 4: Причинно-следственная диаграмма

В таблице 10 приведена таблица решений покрывающая все случаи.
В таблице 11 приведены тесты на основе таблицы решений.

Таблица 10: Таблица решений

Обозначение	1	2	3	4
1	1	1	1	0
2	1	1	0	-
3	1	0	-	-
4	1	0	0	0
5	0	1	0	0
6	0	0	1	1

Таблица 11: Тесты на основе таблицы решений

№	Причины	n	Ожид. рез.	Факт. рез.	Статус
1	1, 2, 3	153	true	true	Пройден
2	1, 2	100	false	false	Пройден
3	1	-10	Ошибка	Ошибка	Пройден
4	1	2000000	Ошибка	Ошибка	Пройден
5	-	10.7	Ошибка	Ошибка	Пройден

3.3.4 Результаты тестирования методом черного ящика

Всего разработано 6 тестов на классы эквивалентности, 8 тестов на граничные значения, 5 тестов на основе таблицы решений (всего 19 тестов). Все тесты успешно пройдены. Это значит, что мне не удалось найти ошибки в программе данным методом.

4 Тестирование программы №2

Автор: Карцев Сергей

Название: Алгоритм Евклида для нахождения наибольшего общего делителя (НОД).

Дано:

- a – первое число;
- b – второе число.

Требуется:

Найти наибольший общий делитель чисел a и b .

Ограничения:

- a является целым числом в диапазоне $[1; 1000]$,
- b является целым числом в диапазоне $[1; 1000]$.

Спецификация

Таблица 12: Спецификация программы нахождения НОД

Входные данные	Ожидаемый результат	Комментарий
$a = 48, b = 18$	6	Обычный случай
$a = 0, b = 25$	Ошибка	Выход за границы допустимых значений
$a = -12, b = 18$	Ошибка	Выход за границы допустимых значений
$a = 0, b = 0$	Ошибка	Выход за границы допустимых значений
$a = 1001, b = 2$	Ошибка	Выход за границы допустимых значений
$a = 10.5, b = 5$	Ошибка	Параметры должны быть целыми числами

4.1 Блок-схема алгоритма

Блок-схема алгоритма приведена на рис. 5.

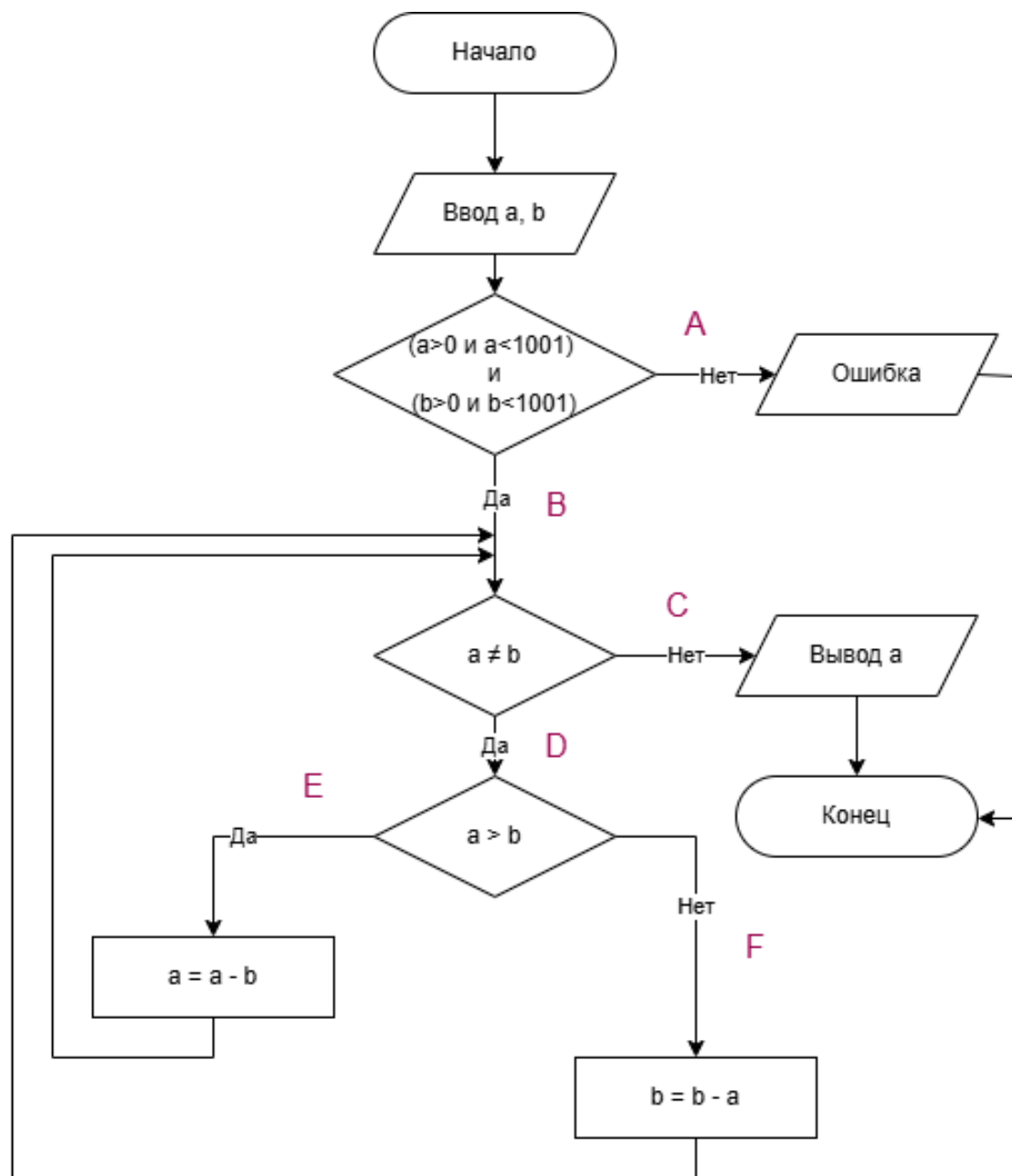


Рис. 5: Блок-схема алгоритма №2

4.2 Тестирование методом белого ящика

В описанной программе можно выделить следующие ключевые условия и соответствующие им ветви.

1. $(a > 0 \ \&\& \ a < 1001 \ \&\& \ b > 0 \ \&\& \ b < 1001)$

(a) Ветвь А: выход из программы с ошибкой.

(b) Ветвь В: продолжение выполнения.

2. $a! = b$

(a) Ветвь С: вывод a , завершение выполнения.

(b) Ветвь D: продолжение выполнения.

3. $a > b$

(a) Ветвь Е: выполнение $a := a - b$, переход на ветвь В.

(b) Ветвь F: выполнение $b := b - a$, переход на ветвь В.

4.2.1 Покрытие операторов

Для покрытия всех операторов достаточно одного теста, который пройдет по основному пути и не вызовет досрочных завершений. В таблице 13 приведены результаты прохождения теста.

Таблица 13: Покрытие операторов

№	a	b	Ожид. рез.	Факт. рез.	Путь	Условия			Статус
						1	2	3	
1	48	18	6	6	BDEDEDFDEC	T	T, F	T, F	Пройден

Тест №1 выполнил все операторы программы.

4.2.2 Покрытие решений

Необходимо составить такой набор тестов, при котором каждое условие в программе примет как истинное, так и ложное значение.

Тесты, покрывающие все решения программы, представлены в табл. 14.

4.2.3 Покрытие условий

В соответствии с критерием покрытия условий количество тестов должно быть таким, чтобы все возможные результаты каждого условия в решении выполнялись как минимум один раз. В табл. 15 приведены тесты для покрытия условий.

Таблица 14: Покрытие решений

№	a	b	Ожид. рез.	Факт. рез.	Путь	Условия			Статус
						1	2	3	
1	48	18	6	6	BDEDFDEC	T	T, F	T, F	Пройден
2	-5	10	Ошибка	Ошибка	A	F	-	-	Пройден

Таблица 15: Покрытие условий

№	a	b	Ожид. рез.	Факт. рез.	Путь	Условия			Статус
						1	2	3	
1	48	18	6	6	BDEDFDEC	(T, T, T, T)	T, F	T, F	Пройден
2	-5	10	Ошибка	Ошибка	A	(F, T, T, T)	-	-	Пройден
3	1001	10	Ошибка	Ошибка	A	(T, F, T, T)	-	-	Пройден
4	10	-5	Ошибка	Ошибка	A	(T, T, F, T)	-	-	Пройден
5	10	1001	Ошибка	Ошибка	A	(T, T, T, F)	-	-	Пройден

4.2.4 Покрытие решений и условий

Набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решений повторяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз. В табл. 16 приведены тесты для покрытия решений и условий.

Таблица 16: Покрытие решений и условий

№	a	b	Ожид. рез.	Факт. рез.	Путь	Условия			Статус
						1	2	3	
1	48	18	6	6	BDEDFDEC	(T, T, T, T)	T, F	T, F	Пройден
2	-5	10	Ошибка	Ошибка	A	(F, T, T, T)	-	-	Пройден
3	1001	10	Ошибка	Ошибка	A	(T, F, T, T)	-	-	Пройден
4	10	-5	Ошибка	Ошибка	A	(T, T, F, T)	-	-	Пройден
5	10	1001	Ошибка	Ошибка	A	(T, T, T, F)	-	-	Пройден

4.2.5 Комбинаторное покрытие условий

Для комбинаторного покрытия условий требуется создать такой набор тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз. В табл. 17 приведены тесты для комбинаторного покрытия условий.

Таблица 17: Комбинаторное покрытие условий

№	a	b	Ожид. рез.	Факт. рез.	Путь	Условия			Статус
						1	2	3	
1	48	18	6	6	BDEDFDEC	(T, T, T, T)	T, F	T, F	Пройден
2	-5	10	Ошибка	Ошибка	A	(F, T, T, T)	-	-	Пройден
3	1001	10	Ошибка	Ошибка	A	(T, F, T, T)	-	-	Пройден
4	10	-5	Ошибка	Ошибка	A	(T, T, F, T)	-	-	Пройден
5	10	1001	Ошибка	Ошибка	A	(T, T, T, F)	-	-	Пройден
6	10.5	30	Ошибка	0.5	BDFDEC	(T, T, T, T)	T, F	T, F	Не пройден
7	30	10.5	Ошибка	0.5	BDEDFC	(T, T, T, T)	T, F	T, F	Не пройден

4.2.6 Результаты тестирования методом белого ящика

В ходе тестирования программы №2 методом белого ящика было разработано: 1 тест для покрытия операторов, 2 теста для покрытия решений, 5 тестов для покрытия условий, 5 тестов для покрытия решений и условий, 7 тестов для комбинаторного покрытия условий.

Среди 20 разработанных тестов 2 теста не было пройдено.

4.3 Тестирование методом чёрного ящика

4.3.1 Разбиение на классы эквивалентности

Из ограничений для программы составлено разбиение на классы эквивалентности, приведенные в табл. 18.

Таблица 18: Классы эквивалентности для a, b

Вх. условие	Допустимые классы	Недопустимые классы
Значения a	Целое число (1), $0 < a < 1001$ (2)	Вещественное значение (3), $a < 1$ (4), $a > 1000$ (5)
Значение b	Целое число (6), $0 < b < 1001$ (7)	Вещественное значение (8), $b < 1$ (9), $b > 1000$ (10)

В таблице 19 приведены тесты, покрывающие все классы эквивалентности.

Таблица 19: Тесты, покрывающие классы эквивалентности

№	a	b	Кл. экв-ти	Ожид. рез.	Факт. рез.	Статус
1	48	18	1, 2, 6, 7	6	6	Пройден
2	18	48	1, 2, 6, 7	6	6	Пройден

3	-48	-18	1, 4, 6, 9	Ошибка	Ошибка	Пройден
4	48	-18	1, 2, 6, 9	Ошибка	Ошибка	Пройден
5	-48	18	1, 4, 6, 7	Ошибка	Ошибка	Пройден
6	25	25	1, 2, 6, 7	25	25	Пройден
7	1001	2	1, 5, 6, 7	Ошибка	Ошибка	Пройден
8	2	1001	1, 2, 6, 10	Ошибка	Ошибка	Пройден
9	10.5	10	2, 3, 6, 7	Ошибка	0.5	Не пройден
10	10	10.5	1, 2, 7, 8	Ошибка	0.5	Не пройден

При покрытии классов эквивалентности два теста №9,10 не прошли тестирование:

- в тесте №9 при вводе вещественного числа a программа должна завершаться с ошибкой, но она продолжает обработку вещественного числа;
- в тесте №10 при вводе вещественного числа b программа должна завершаться с ошибкой, но она продолжает обработку вещественного числа.

4.3.2 Анализ граничных условий

Согласно спецификации, входные значения должны являться целыми числами, находящимися в диапазоне $[1, 1000]$. Значит, можно выделить следующие граничные условия:

- $a \leq 1$;
- $a \geq 1000$;
- $b \leq 1$;
- $b \geq 1000$.

Для каждой из границ определим тесты, соответствующие:

- граничному целому числу (верхнему/нижнему);
- целому числу, выходящему за верхнюю/нижнюю границу;
- дробному числу, на 0.001 выходящему за верхнюю/нижнюю границу.

Для каждой границы определены тесты, приведенные в таблице 20.

Таблица 20: Тесты, покрывающие проверку граничных условий

№	a	b	Ожид. рез.	Факт. рез.	Статус
1	1	1000	1	1	Пройден
2	1000	1	1	1	Пройден
3	1	1	1	1	Пройден
4	1000	1000	1000	1000	Пройден
5	0	1000	Ошибка	Ошибка	Пройден
6	1000	0	Ошибка	Ошибка	Пройден
7	1001	1	Ошибка	Ошибка	Пройден
8	1	1001	Ошибка	Ошибка	Пройден
9	0.999	5	Ошибка	0.001	Не пройден
10	1	1000.001	Ошибка	0.001	Не пройден

При тестировании не были пройдены тесты №9,10, так как программа вместо того, чтобы завершиться с ошибкой, продолжает обрабатывать вещественные числа и работает до тех пор, пока не вернет результат.

4.3.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введены следующие обозначения.

Причины.

1. a – целое.
2. $a > 0$ И $a < 1001$.
3. b – целое.
4. $b > 0$ И $b < 1001$.

Промежуточные причины

11. a – целое И $a > 0$ И $a < 1001$.
12. b – целое И $b > 0$ И $b < 1001$.

Следствия.

21. Программа выводит сообщение об ошибке.
22. Программа возвращает результат.

На рис. 6 приведена причинно-следственная диаграмма для рассматриваемой программы.

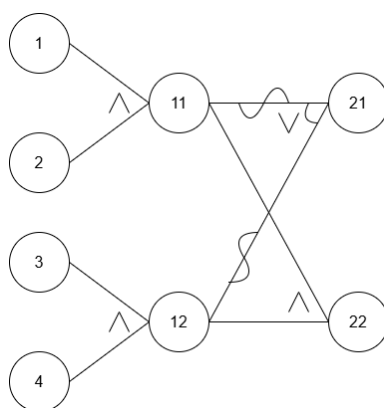


Рис. 6: Причинно-следственная диаграмма

В таблице 21 приведена таблица решений покрывающая все случаи.

Таблица 21: Таблица решений

Обозначение	1	2	3	4	5
1	1	0	1	1	1
2	1	1	0	1	1
3	1	1	1	0	1
4	1	1	1	1	0
11	1	0	0	1	1
12	1	1	1	0	0
21	0	1	1	1	1
22	1	0	0	0	0

В таблице 22 приведены тесты, составленные на основе таблицы решений.

Таблица 22: Тесты на основе таблицы решений

№	Причины	a	b	Ожид. рез.	Факт. рез.	Статус
1	1, 2, 3, 4	12	6	6	6	Пройден
2	2, 3, 4	10.5	10	Ошибка	0.5	Не пройден
3	1, 3, 4	10001	10	Ошибка	Ошибка	Пройден
4	1, 2, 4	10	10.5	Ошибка	0.5	Не пройден
5	1, 2, 3	10	10001	Ошибка	Ошибка	Пройден

Было обнаружено два непройденных теста №2,4:

- в тесте №2 при вводе вещественного числа a программа должна завершаться с ошибкой, но она продолжает обработку вещественного числа;
- в тесте №4 при вводе вещественного числа b программа должна завершаться с ошибкой, но она продолжает обработку вещественного числа.

4.3.4 Результаты тестирования методом черного ящика

При тестировании программы №2 методом черного ящика было разработано 10 тестов для покрытия всех допустимых и недопустимых классов эквивалентности, 10 тестов для проверки граничных значений и 5 тестов на основе таблицы решений. Из 25 составленных тестов 6 тестов было не пройдено.

Заключение

В ходе выполнения данной лабораторной работы были изучены методы тестирования «белым ящиком» и «черным ящиком».

В результате тестирования программы №1 методом «белого ящика» все 5 тестовых наборов оказались пройдены. Тестирование методом «черного ящика» так же показало, что все тесты пройдены. Мне не удалось достичь цели тестирования и подобрать такие наборы тестов, при которых ошибки в программе были бы обнаружены.

В результате тестирования программы №2 методом «черного ящика» выявлено 6 непройденных тестов из 25, что показало эффективность этого метода тестирования для данной программы. Тестирование методом «белого ящика» выявило 2 непройденных теста.

По итогу выполненной работы, я сделала вывод, что оба метода тестирования могут быть полезны для выявления ошибок в программе, однако, какой из методов окажется эффективнее заранее сказать не получится, поэтому наиболее правильным решением будет проводить тестирование обоими методами.